

Predicción del riesgo de ictus

Machine Learning · Bootcamp Data Science / AI Developer

Proyecto 8 · 2026

Primera parte — Idea de negocio

Presentación al cliente

Propósito · Problema · Público objetivo

- Propósito: modelo de ML que, a partir de datos clínicos de un paciente, devuelve la probabilidad de sufrir un ictus.
- Problema que soluciona: cribado temprano del riesgo de ictus, donde la clase positiva (ictus) es minoritaria (4.98 %).
- Público objetivo: personal sanitario, equipos de desarrollo y pacientes en campañas de prevención.

User stories

- Como personal sanitario, quiero conocer la probabilidad de ictus de un paciente, para priorizar la atención.
- Como desarrollador/a, quiero invocar la predicción desde un servicio HTTP (API), para integrarla en una aplicación.
- Como paciente, quiero una interfaz sencilla que muestre claramente mi nivel de riesgo.

Tecnologías utilizadas

- EDA: Pandas, Seaborn, Matplotlib, missingno (notebooks 01–07).
- Modelos: scikit-learn, CatBoost, XGBoost, imbalanced-learn.
- Productivización: FastAPI, Uvicorn, Pydantic (API REST + frontend).
- Base de datos: PostgreSQL (Docker) / SQLite (local).
- Tooling: uv, Makefile, pytest, Git, Docker Compose.

Estructura del código

- BACKEND/ — API FastAPI (main.py), adaptador BD (db.py) y frontend (static/).
- scripts/ — entrenamiento (train_*), comparativa (compare) y CLI (predict_cli).
- models/ — modelo final (catboost_final.pkl) + comparativos + informe.
- tests/ — suite de tests pytest + informe.
- SDD/ — decisiones de diseño (D1–D13).
- dockerfile + docker-compose.yml — contenedor y orquestación.

Uso de servidores de despliegue

- Planificado en Render (Web Service + PostgreSQL).
- Ejecución actual en local: `make api` → `http://127.0.0.1:8000` (frontend + API).
- El modelo y la app están listos para contenerizarse y desplegarse.

Herramientas de nube y contenedores

- Nube (AWS/GCP/Azure/S3/Firebase): No se utiliza — todo se ejecuta en local/Postgres.
- Contenedores (Docker): Configurado — docker-compose con 2 servicios (db + api).
- Orquestación: docker-compose up -d --build (make docker-up).
- Kubernetes: No se utiliza.

Repositorio y CI

- Repo: GitHub — Bootcamp-IA-MAD-P7/Proyecto-8-DataScience-Veru.
- Ramas: feature → dev → main (EDA, models, informe, API, database, frontend, docker).
- Commits descriptivos en español (feat/, fix/, chore/, docs/).
- GitHub Actions: No configurado — verificación manual con make test (24 tests).

Flujo de usuario (userflow)

- Vía CLI: `python scripts/predict_cli.py --age ... --gender ...` → probabilidad + veredicto.
- Vía API: `POST /predict (JSON)` → {probabilidad_ictus, clase, riesgo}.
- Vía Web: `GET /` → formulario con 10 campos → gauge + veredicto + historial.
- Historial: `GET /predictions` (guardado en BD).

Segunda parte — Parte técnica

Presentación al equipo de desarrollo

Implementaciones importantes

- Pipeline de preprocesado + modelo (evita data leakage, D6).
- Desbalance: `scale_pos_weight` / `class_weight` / SMOTE (D2).
- Overfitting: $|\text{train-test}| \leq 5$ puntos (D3.3).
- Validación cruzada `StratifiedKFold(5)` (D5.1) y `GridSearchCV` (D5.2).
- CLI + API REST + Frontend web + BD historial + Docker.

Diseño de la base de datos (D11)

- Origen 1: data/stroke_dataset.csv (entrenamiento, solo lectura).
- Origen 2: tabla predictions en PostgreSQL (producción) o SQLite (local).
- Entidad única: prediction (id PK, 10 features + probabilidad, clase, riesgo, created_at).
- Sin relaciones ni claves foráneas (modelo sobre paciente puntual).
- Endpoints: POST /predict (guarda) y GET /predictions (historial).

Paradigma, estructura y patrones

- Desarrollo funcional por script, OOP donde aporta (Pydantic, Pipeline).
- Patrón: Pipeline de scikit-learn (preprocesado + modelo).
- Bootstrap: BACKEND/main.py carga el modelo al arrancar (joblib.load).
- Config por constantes de módulo (THRESHOLD, RANDOM_STATE, FEATURES).

Convenciones, repositorio y documentación

- Naming: scripts snake_case (train_, predict_, compare_), funciones verbos, clases UpperCamelCase.
- Modelos: make train-* → models/<modelo>.pkl; constantes en MAYÚSCULAS_SNAKE.
- Docs: README.md, SDD (D1–D13), models/INFORME_MODELOS.md, tests/INFORME_TESTS.md.
- API servidor: /health, /predict, /predictions + Swagger /docs.

Modelo final y cierre

- Modelo: CatBoost regularizado (config 'med').
- Métricas en test: Recall 0.86 · Precision 0.146 · F1 0.250.
- Overfitting: $|\text{train} - \text{test}| \leq 1.4$ puntos (límite 5). 
- Tests: 24 tests pasando (make test).
- Conclusiones: age domina la importancia; desbalance limita la precisión.
- Vías futuras: más variables, despliegue completo en Render.